

CS311- Design & Analysis of Algorithms

Divide and Conquer :

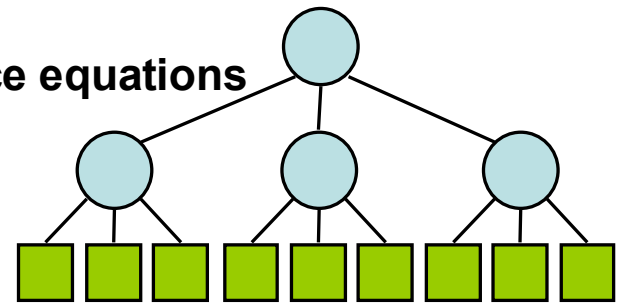
Merge Sort & Quick Sort

Divide-and-Conquer

The most-well known algorithm design strategy:

1. ***Divide*** the problem into sub-problems that are similar to the original but smaller in size
2. ***Conquer*** the sub-problems by solving them recursively. If they are small enough, just solve them in a straightforward manner
3. ***Combine*** the solutions to create a solution to the original problem

- Analysis can be done using **recurrence equations**



A general divide-and-conquer algorithm

Step 1: If the problem size is small, solve this problem directly; otherwise, split the original problem into 2 sub-problems with equal sizes.

Step 2: Recursively solve these 2 sub-problems by applying this algorithm.

Step 3: Merge the solutions of the 2 sub-problems into a solution of the original problem.

Classic example

Sorting algorithms

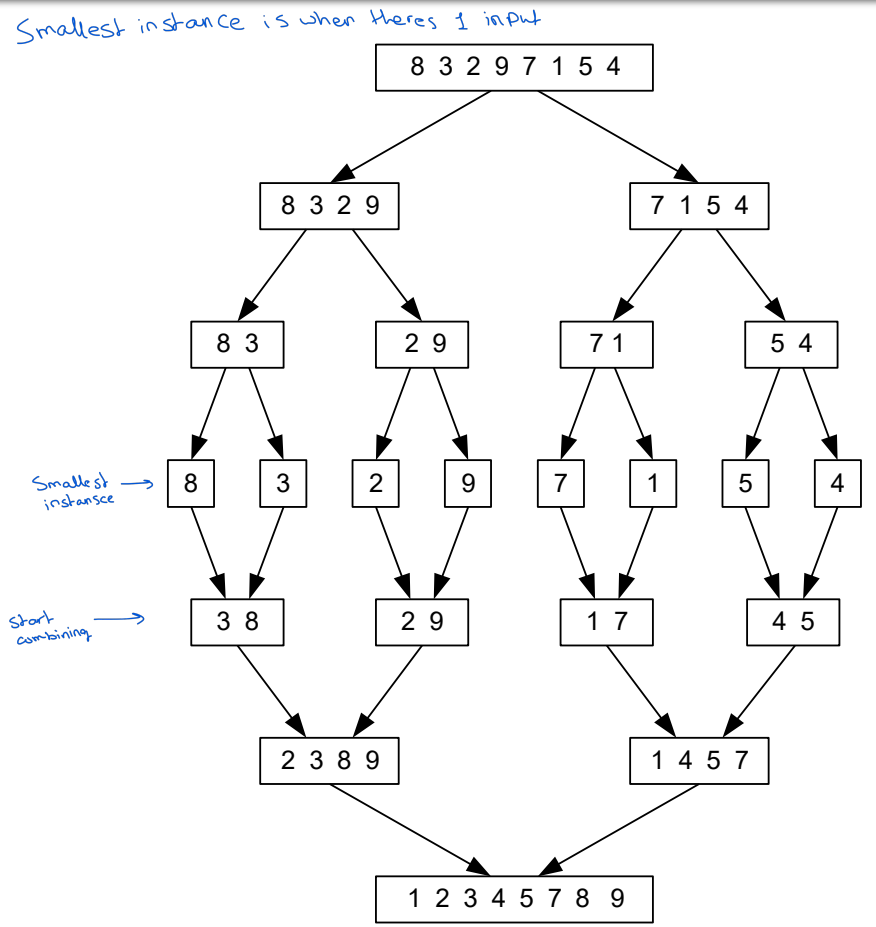
- Mergesort
- Quicksort

An Example: Merge Sort

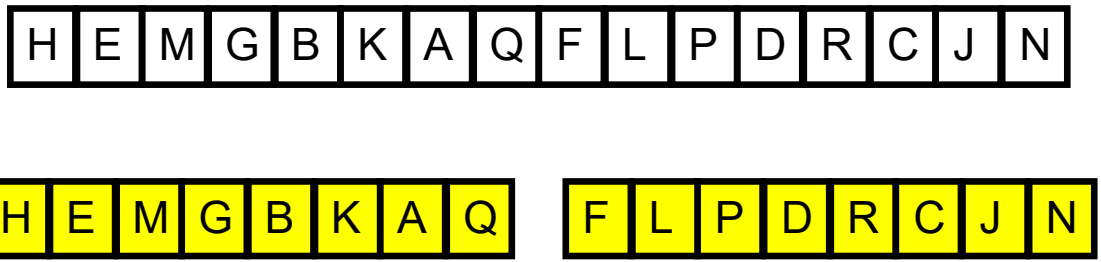
***Sorting Problem*:** Sort a sequence of *n* elements into increasing order.

- ***Divide*:** Divide the *n*-element sequence to be sorted into two subsequences of *n/2* elements each
- ***Conquer:*** Sort the two subsequences recursively using merge sort.
- ***Combine*:** Merge the two sorted subsequences to produce the sorted answer.

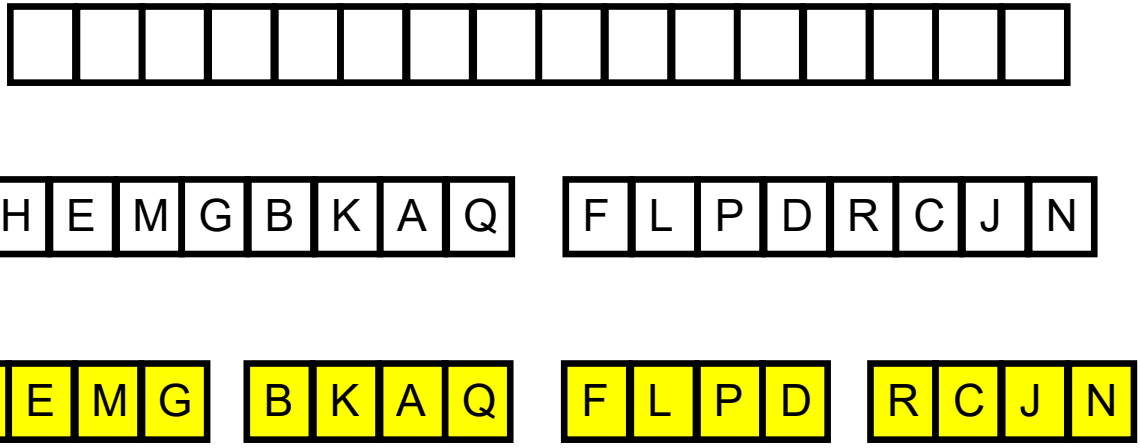
Mergesort Example



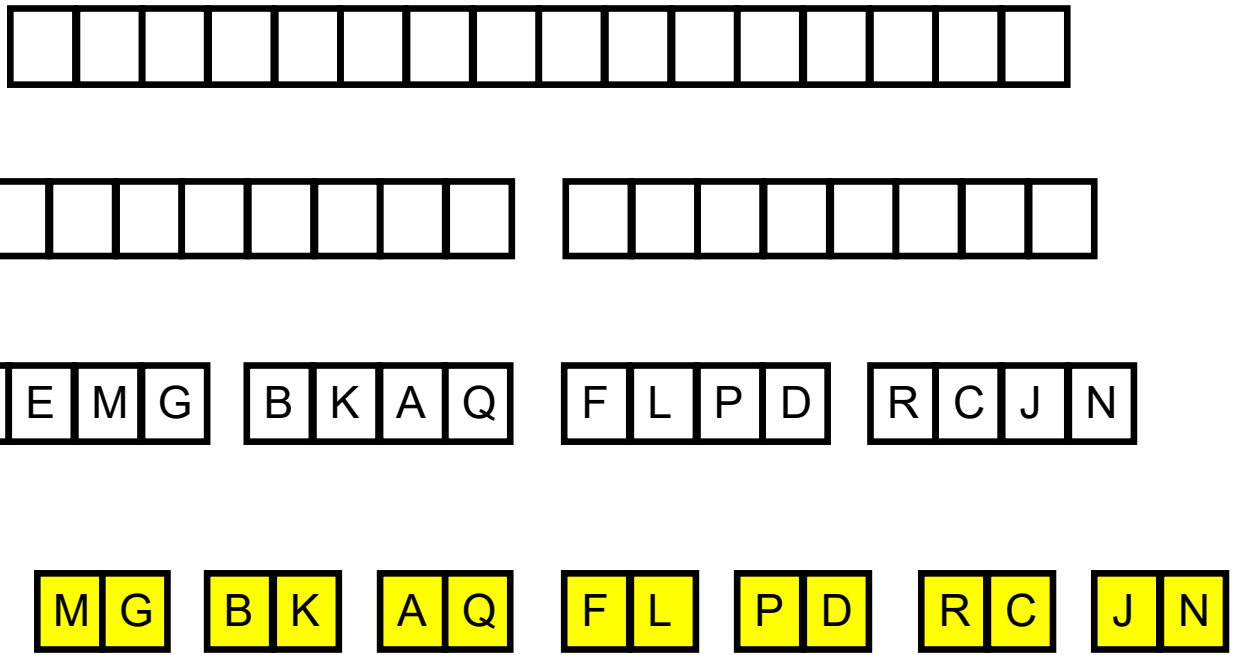
Subdivide the sorting task



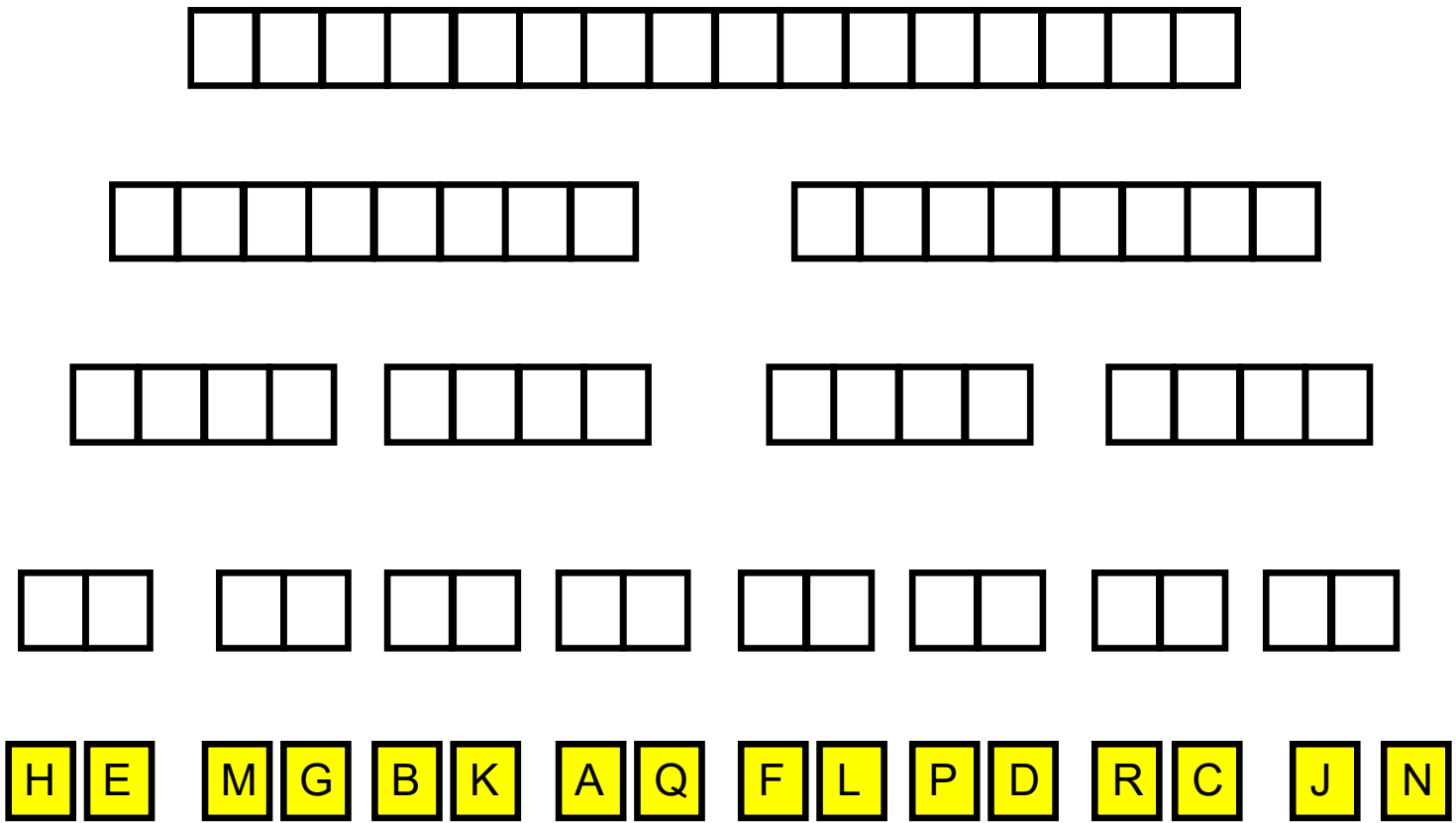
Subdivide again



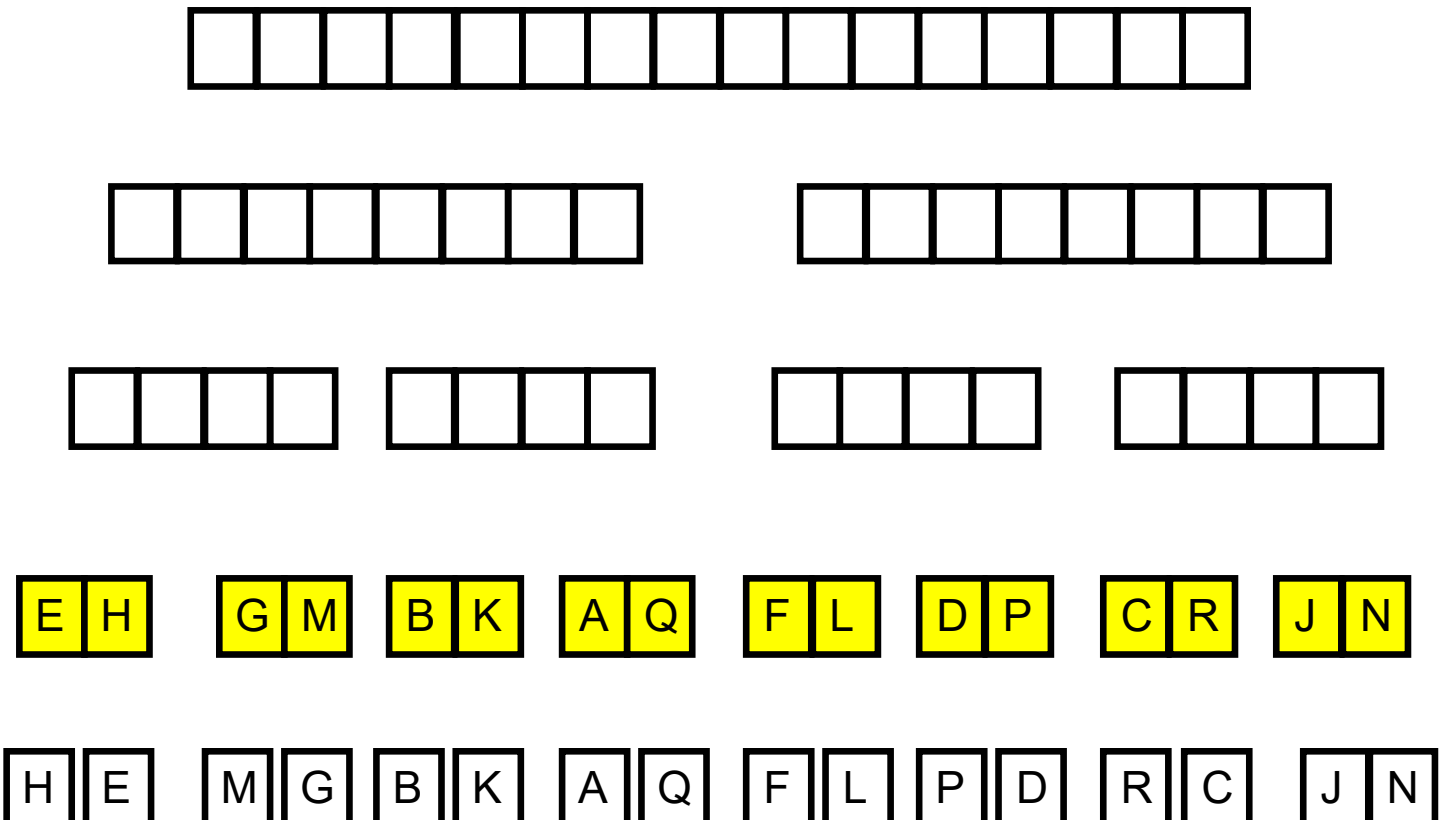
And again



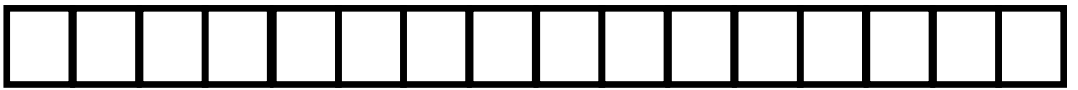
And one last time

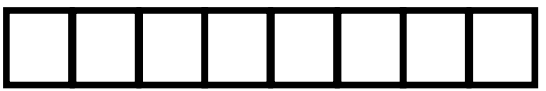


Now merge



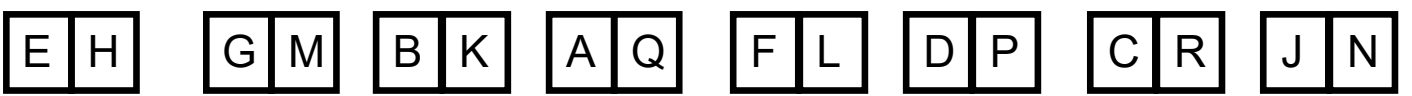
And merge again



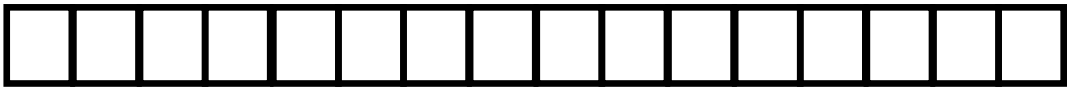








And again















And one last time







Done!



Pseudocode of Mergesort

- MergeSort (Array(First..Last))

```
Begin
    If Array contains only one element Then
        Return Array
    Else
        Middle= ((Last + First)/2) rounded down to the nearest integer
        LeftHalfArray = MergeSort(Array(First..Middle))
        RightHalfArray = MergeSort(Array(Middle+1..Last))
        ResultArray = Merge(LeftHalfArray, RightHalfArray)
        Return ResultArray
    EndIf
End MergeSort
```

$T(n) = 2T(\frac{n}{2}) + n$

$\Theta(n \log n)$

Pseudocode of Merge

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$ } c
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ j \leftarrow j+1$
    $k \leftarrow k+1$
n
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

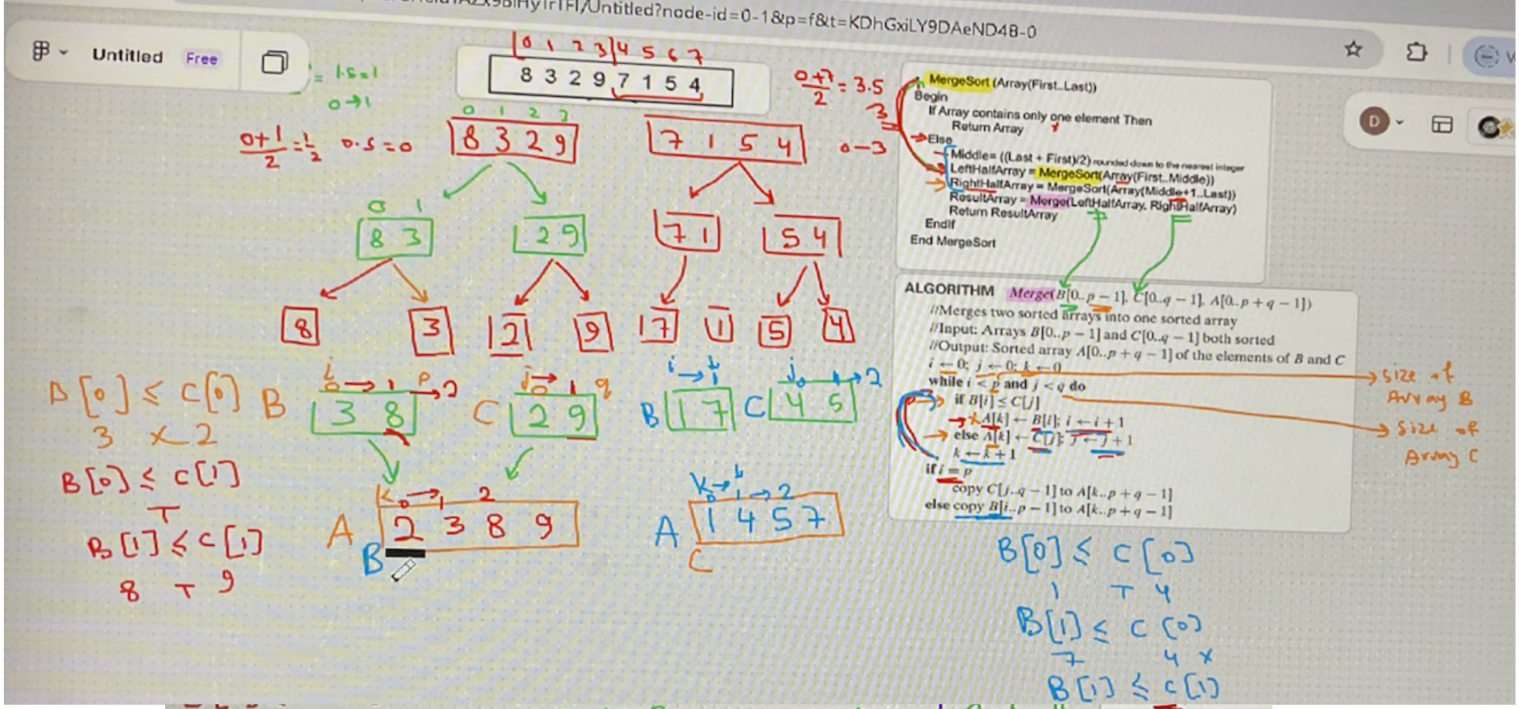
Analysis of Merge Sort

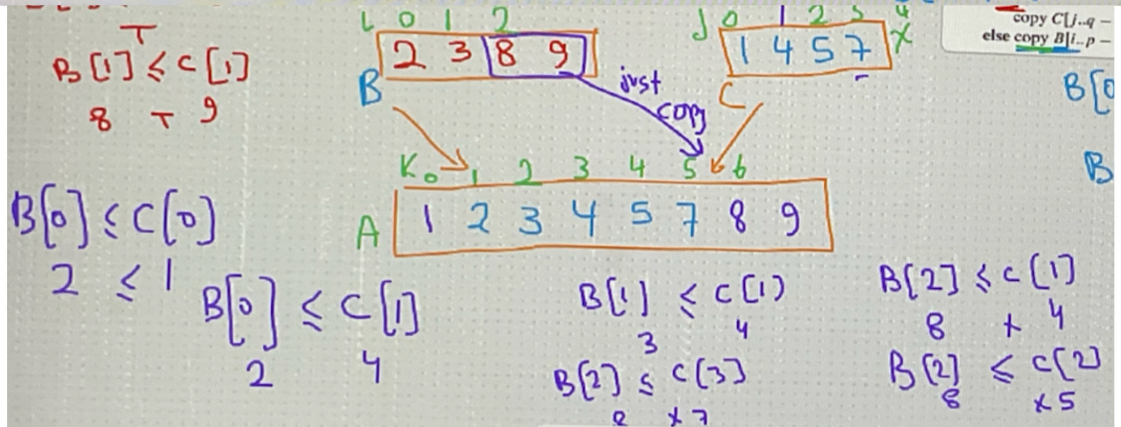


- Running time $T(n)$ of Merge Sort:
- Divide: computing the middle takes $\Theta(1)$
- Conquer: solving 2 subproblems takes $2T(n/2)$
- Combine: merging n elements takes $\Theta(n)$
- Total:

$$T(n) = \Theta(1) \quad \text{if } n = 1$$
$$T(n) = 2T(n/2) + \Theta(n) \quad \text{if } n > 1$$

$\Rightarrow$ *Solve the recurrence*

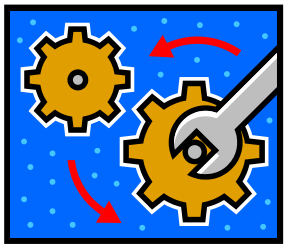

Untitled
Free
8 3 2 9 7 1 5 4
MergeSort (Array(First..Last))
Begin
If Array contains only one element Then
Return Array
Else
Middle= ((Last + First)/2) rounded down to the nearest integer
LeftHalfArray = MergeSort(Array(First..Middle))
RightHalfArray = MergeSort(Array(Middle+1..Last))
ResultArray = Merge(LeftHalfArray, RightHalfArray)
Return ResultArray
EndIf
End MergeSort
ALGORITHM Merge(B[0..p − 1], C[0..q − 1], A[0..p + q − 1])
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted
//Output: Sorted array A[0..p + q − 1] of the elements of B and C
i ← 0; j ← 0; k ← 0
while i < p and j < q do
if B[i] ≤ C[j]
A[k] ← B[i]; i ← i + 1
else A[k] ← C[j]; j ← j + 1
k ← k + 1
if i = p
copy C[j..q − 1] to A[k..p + q − 1]
else copy B[i..p − 1] to A[k..p + q − 1]
size of Array B
size of Array C


B[1] ≤ C[1]
8 T 9
2 3 8 9
1 4 5 7
just copy
1 2 3 4 5 7 8 9
B[0] ≤ C[0]
2 ≤ 1
B[0] ≤ C[1]
2 4
B[1] ≤ C[1]
3 4
B[2] ≤ C[3]
8 ≤ 7
B[2] ≤ C[1]
8 ≤ 4
B[2] ≤ C[2]
8 ≤ 5

Recurrence Equation Analysis

- In the iterative substitution, we iteratively apply the recurrence equation to itself and see if we can find a pattern:

$$
\begin{aligned}
T(n) &= 2T(n/2) + bn \\
&= 2(2T(n/2^2)) + b(n/2)) + bn \\
&= 2^2 T(n/2^2) + 2bn \\
&= 2^3 T(n/2^3) + 3bn \\
&= 2^4 T(n/2^4) + 4bn \\
&= \ldots \\
&= 2^i T(n/2^i) + ibn \quad \text{i is k}
\end{aligned}
$$



$$T(n) = bn + bn \log n \quad \text{b is a constant}$$

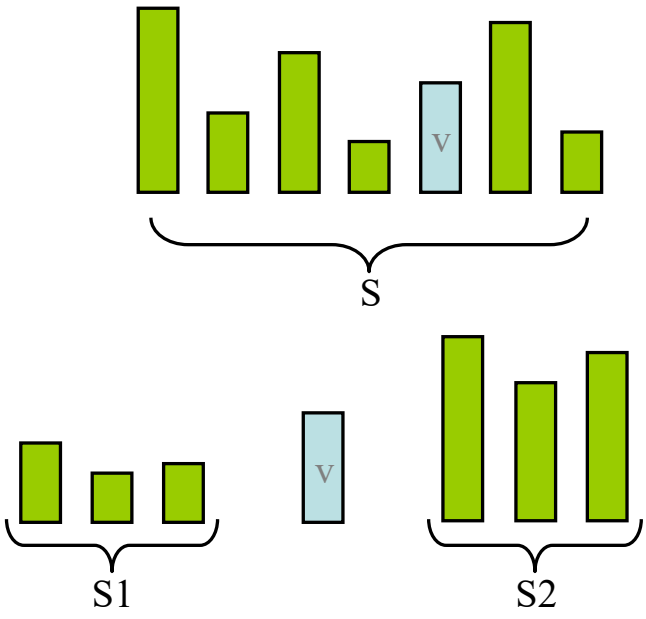
- Note that base, T(n)=b, case occurs when $2^i = n$. That is, i = log n.
- So,
- Thus, T(n) is O(n log n).

Quick Sort

- Developed by Hoare (1962)
- Fastest known sorting algorithm in practice
- Divide-and-conquer algorithm, similar to Mergesort:
  - Sort is accomplished by dividing the array into two partitions
  - Then sorting each partition recursively

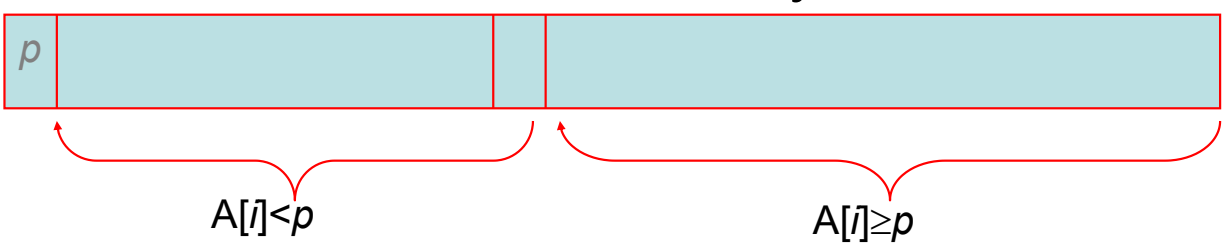
Quicksort

- Divide:
  - Pick any element (***pivot***) v in S
  - Partition S – {v} into two disjoint groups
    S1 = {x ∈ S – {v} | x < v}
    S2 = {x ∈ S – {v} | x ≥ v}
- Conquer : recursively sort S1 and S2
- Combine : the sorted S1 (by the time returned from recursion), followed by v, followed by the sorted S2 (i.e., nothing extra needs to be done)



Quicksort

- Select a *pivot* (partitioning element)
- Rearrange the list so that all the elements in the positions before the pivot are smaller than to the pivot and those after the pivot are larger than or equal to the pivot
- Exchange the pivot with the last element in the first (i.e., <) sublist – the pivot is now in its final position
- Sort the two sublists recursively



Quicksort: Pseudocode

- To sort a[left...right]:

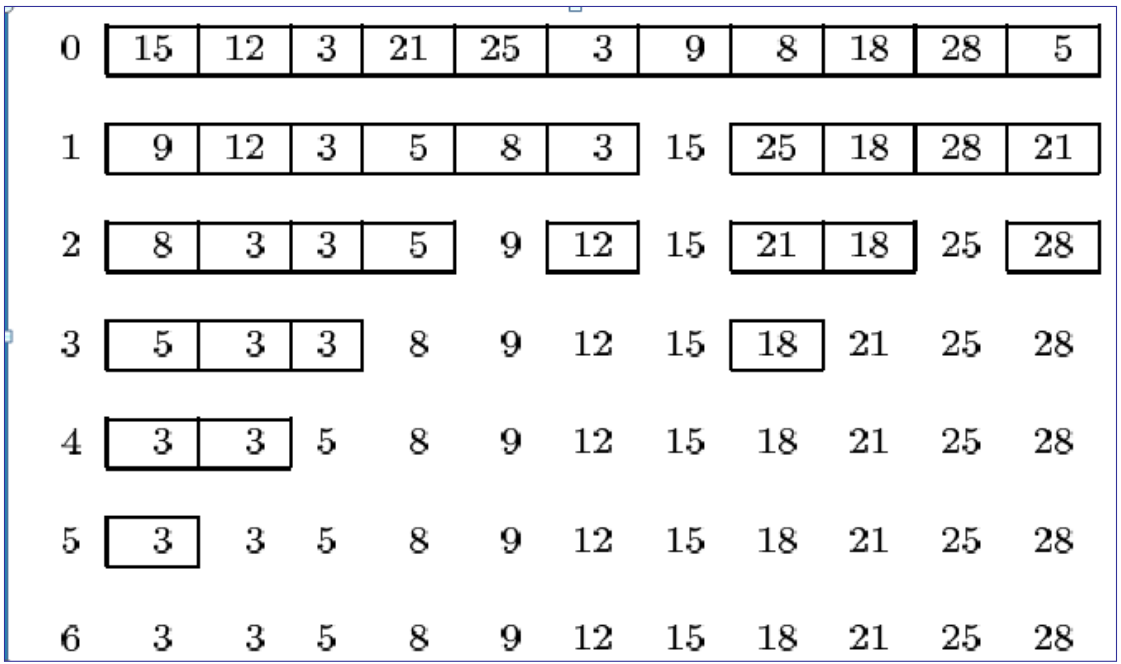
```
if left < right:
    Partition a[left...right] such that:
            all a[left...p-1] are less than a[p], and
            all a[p+1...right] are >= a[p]
    Quicksort a[left...p-1]
    Quicksort a[p+1...right]
Terminate
```

Partitioning (Quicksort)

- Choose an array value (say, the first) to use as the pivot
- Starting from the left end, find the first element that is greater than or equal to the pivot
- Searching backward from the right end, find the first element that is less than the pivot
- Interchange (swap) these two elements
- Repeat, searching from where we left off, until done

Quicksort: Example

- Recursive implementation with the left most array entry selected as the pivot element. (first element)



Example of partitioning

- choose pivot: **4 3 6 9 2 4 3 1 2 1 8 9 3 5 6**
- search: **4 3 6 9 2 4 3 1 2 1 8 9 3 5 6**
- swap: **4 3 3 9 2 4 3 1 2 1 8 9 6 5 6**
- search: **4 3 3 9 2 4 3 1 2 1 8 9 6 5 6**
- swap: **4 3 3 1 2 4 3 1 2 9 8 9 6 5 6**
- search: **4 3 3 1 2 4 3 1 2 9 8 9 6 5 6**
- swap: **4 3 3 1 2 2 3 1 4 9 8 9 6 5 6**
- search: **4 3 3 1 2 2 3 1 4 9 8 9 6 5 6**
- swap with pivot: **1 3 3 1 2 2 3 4 4 9 8 9 6 5 6**

Best-case Analysis



- What will be the best case?
  - Partition is perfectly balanced: We cut the array size in half each time.
  - Pivot is always in the middle

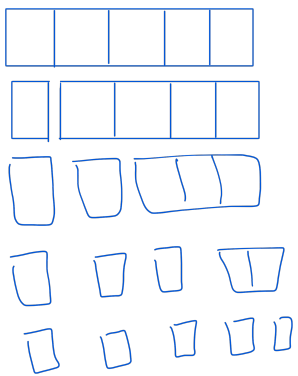
$$
\begin{aligned}
T(n) &= 2T(n/2) + bn \\
&= 2(2T(n/2^2)) + b(n/2)) + bn \\
&= 2^2 T(n/2^2) + 2bn \\
&= 2^3 T(n/2^3) + 3bn \\
&= 2^4 T(n/2^4) + 4bn \\
&= \ldots \\
&= 2^i T(n/2^i) + ibn
\end{aligned}
$$

Thus, T(n) is O(n log n).

Worst case



- In the worst case, partitioning always divides the size n array into these three parts: $O(n^2)$
  - A length one part, containing the pivot itself
  - A length zero part, and
  - A length n-1 part, containing everything else



when the array is already sorted or in the opposite way like sorted ascendingly or descendingly

ex:

10	9	8	7	6	5	4	3	2	1

Worst-Case Analysis



- What will be the worst case?
  - The pivot is the smallest element, all the time
  - Partition is always unbalanced

$$
\begin{aligned}
T(N) &= T(N-1) + cN \\
T(N-1) &= T(N-2) + c(N-1) \\
T(N-2) &= T(N-3) + c(N-2) \\
&\vdots \\
T(2) &= T(1) + c(2) \\
T(N) &= T(1) + c\sum_{i=2}^{N} i = O(N^2)
\end{aligned}
$$

Worst case for Quicksort



- In the worst case, recursion may be n levels deep (for an array of size n)
- But the partitioning work done at each level is still n
- So worst case for Quicksort is $O(n^2)$
- When does this happen?
  - There are many arrangements that *could* make this happen
  - Here are two common cases:
    - When the array is already sorted
    - When the array is *inversely* sorted (sorted in the opposite order)

Picking a better pivot

- Before, we picked the *first* element of the subarray to use as a pivot
  - If the array is already sorted, this results in $O(n^2)$ behavior
  - It's no better if we pick the *last* element
- We could do an *optimal* quicksort (guaranteed $O(n \log n)$) if we always picked a pivot value that exactly cuts the array in half
  - Such a value is called a median: half of the values in the array are larger, half are smaller
  - Unfortunately, its hard to calculate median quickly, without sorting first!

Picking a better pivot

- So find the approximate median
  - Pivot = median of the left-most, right-most and center element of the array `S`
  - Solves the problem of sorted input

- Example: Median-of-three Partitioning (cut in the middle each time and use that as a pivot)
  - Let input `S = {0, 1,2,3,4,5,6,7,8,9}`
  - `left=0` and `S[left] = 0`
  - `right=9` and `S[right] = 9`
  - `center = (left+right)/2 = 4` and `S[center] = 4`

  - Pivot
    - = Median of `S[left], S[right],` and `S[center]`
    - = median of 0, 9, and 4
    - `= 4`

Quicksort (exercice)

- Apply quicksort to sort the list 7 2 9 10 5 4

